

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №5
«Расстояние Левенштейна на C#»

Выполнил:
Студент группы ИУ5-35Б
Гремина Анна

Проверил:
Нардид А. Н.

2025 г.

Задание

Разработать программу, реализующую вычисление расстояния Левенштейна с использованием алгоритма Вагнера-Фишера.

1. Программа должна быть разработана в виде библиотеки классов на языке C#.
2. Использовать самый простой вариант алгоритма без оптимизации.
3. Дополнительно возможно реализовать вычисление расстояния Дamerau-Левенштейна (с учетом перестановок соседних символов).
4. Модифицировать предыдущую лабораторную работу, вместо поиска подстроки используется вычисление расстояния Левенштейна.
5. Предусмотреть отдельное поле ввода для максимального расстояния. Если расстояние Левенштейна между двумя строками больше максимального, то строки считаются несовпадающими и не выводятся в список результатов.

Листинг кода

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4
5  namespace LevenshteinLibrary
6  {
7      public class LevenshteinCalculator
8      {
9          public static int Calculate(string source, string target)
10         {
11             if (string.IsNullOrEmpty(source))
12                 return string.IsNullOrEmpty(target) ? 0 : target.Length;
13
14             if (string.IsNullOrEmpty(target))
15                 return source.Length;
16
17             int sourceLength = source.Length;
18             int targetLength = target.Length;
19
20             int[,] matrix = new int[sourceLength + 1, targetLength + 1];
21
22             for (int i = 0; i <= sourceLength; i++)
23                 matrix[i, 0] = i;
24
25             for (int j = 0; j <= targetLength; j++)
26                 matrix[0, j] = j;
27
28             for (int i = 1; i <= sourceLength; i++)
29             {
30                 for (int j = 1; j <= targetLength; j++)
31                 {
32                     int cost = (source[i - 1] == target[j - 1]) ? 0 : 1;
33
34                     matrix[i, j] = Math.Min(
35                         Math.Min(
36                             matrix[i - 1, j] + 1,
37                             matrix[i, j - 1] + 1),
38                         matrix[i - 1, j - 1] + cost);
39                 }
40             }
41
42             return matrix[sourceLength, targetLength];
43         }
44
45         public static int CalculateWithTransposition(string source, string target)
46         {
47             if (string.IsNullOrEmpty(source))
48                 return string.IsNullOrEmpty(target) ? 0 : target.Length;
49
50             if (string.IsNullOrEmpty(target))
51                 return source.Length;
52
53             int sourceLength = source.Length;
54             int targetLength = target.Length;
55
56             int[,] matrix = new int[sourceLength + 1, targetLength + 1];
57
58             for (int i = 0; i <= sourceLength; i++)
59                 matrix[i, 0] = i;
60
61             for (int j = 0; j <= targetLength; j++)
62                 matrix[0, j] = j;
63
64             for (int i = 1; i <= sourceLength; i++)
65             {
66                 for (int j = 1; j <= targetLength; j++)
67                 {
68                     int cost = (source[i - 1] == target[j - 1]) ? 0 : 1;
69
70                     matrix[i, j] = Math.Min(
71                         Math.Min(
72                             matrix[i - 1, j] + 1,
73                             matrix[i, j - 1] + 1),
74                         matrix[i - 1, j - 1] + cost);
75                 }
76             }
77
78             return matrix[sourceLength, targetLength];
79         }
80     }
81 }
```

```

70         matrix[i, j] = Math.Min(
71             Math.Min(
72                 matrix[i - 1, j] + 1,
73                 matrix[i, j - 1] + 1),
74             matrix[i - 1, j - 1] + cost);
75
76         if (i > 1 && j > 1 &&
77             source[i - 1] == target[j - 2] &&
78             source[i - 2] == target[j - 1])
79         {
80             matrix[i, j] = Math.Min(matrix[i, j], matrix[i - 2, j - 2] + cost);
81         }
82     }
83 }
84
85 return matrix[sourceLength, targetLength];
86 }
87
88 public static bool IsWithinMaxDistance(string source, string target, int maxDistance)
89 {
90     int distance = Calculate(source, target);
91     return distance <= maxDistance;
92 }
93
94 public static int[,] GetDistanceMatrix(string source, string target)
95 {
96     if (string.IsNullOrEmpty(source) && string.IsNullOrEmpty(target))
97         return new int[1, 1] { { 0 } };
98
99     int sourceLength = source?.Length ?? 0;
100    int targetLength = target?.Length ?? 0;

```

```

101
102    int[,] matrix = new int[sourceLength + 1, targetLength + 1];
103
104    for (int i = 0; i <= sourceLength; i++)
105        matrix[i, 0] = i;
106
107    for (int j = 0; j <= targetLength; j++)
108        matrix[0, j] = j;
109
110    if (sourceLength > 0 && targetLength > 0 && source != null && target != null)
111    {
112        for (int i = 1; i <= sourceLength; i++)
113        {
114            for (int j = 1; j <= targetLength; j++)
115            {
116                int cost = (source[i - 1] == target[j - 1]) ? 0 : 1;
117
118                matrix[i, j] = Math.Min(
119                    Math.Min(
120                        matrix[i - 1, j] + 1,
121                        matrix[i, j - 1] + 1),
122                    matrix[i - 1, j - 1] + cost);
123            }
124        }
125    }
126
127    return matrix;
128 }
129
130 public static string MatrixToString(string source, string target)
131 {
132     int[,] matrix = GetDistanceMatrix(source, target);
133     int sourceLength = source?.Length ?? 0;
134     int targetLength = target?.Length ?? 0;

```

```

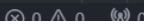
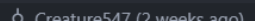
135
136     string result = "Матрица расстояний Левенштейна:\n\n";
137     result += "    ";
138
139     if (target != null)
140     {
141         for (int j = 0; j < targetLength; j++)
142             result += $"{target[j],3}";
143     }
144
145     result += "\n";
146
147     for (int i = 0; i <= sourceLength; i++)
148     {
149         if (i == 0)
150             result += "    ";
151         else if (source != null)
152             result += $"{source[i - 1],2}";
153
154         for (int j = 0; j <= targetLength; j++)
155         {
156             result += $"{matrix[i, j],3}";
157         }
158         result += "\n";
159     }
160
161     return result;
162 }
163 }

```

```

164
165 class Program
166 {
167     static void Main(string[] args)
168     {

```


[Error resolving NBT](#)

[Creature547 \(2 weeks ago\)](#)
 Ln 243, Col 39

```

169     Console.WriteLine("=== Лабораторная работа №5: Расстояние Левенштейна ===\n");
170
171     while (true)
172     {
173         ShowMenu();
174         string? choice = Console.ReadLine();
175
176         switch (choice)
177         {
178             case "1":
179                 SimpleCalculation();
180                 break;
181             case "2":
182                 WithMaxDistance();
183                 break;
184             case "3":
185                 ShowMatrix();
186                 break;
187             case "4":
188                 BatchTest();
189                 break;
190             case "5":
191                 Console.WriteLine("Выход из программы...");
192                 return;
193             default:
194                 Console.WriteLine("Неверный выбор. Попробуйте снова.\n");
195                 break;
196         }
197     }
198 }
199
200 static void ShowMenu()
201 {
202     Console.WriteLine("\n--- МЕНЮ ---");

```

```

203 Console.WriteLine("1. Простое вычисление расстояния");
204 Console.WriteLine("2. Проверка ☐ максимальным порогом");
205 Console.WriteLine("3. Показать матрицу расстояний");
206 Console.WriteLine("4. Пакетное тестирование");
207 Console.WriteLine("5. Выход");
208 Console.Write("\nВыберите действие: ");
209 }
210
211 static void SimpleCalculation()
212 {
213     Console.WriteLine("\n--- Простое вычисление расстояния ---");
214     Console.Write("Введите первую строку: ");
215     string? str1 = Console.ReadLine();
216     Console.Write("Введите вторую строку: ");
217     string? str2 = Console.ReadLine();
218
219     if (str1 == null || str2 == null)
220     {
221         Console.WriteLine("Ошибка: некорректный ввод");
222         return;
223     }
224
225     int distance = LevenshteinCalculator.Calculate(str1, str2);
226     int distanceWithTransposition = LevenshteinCalculator.CalculateWithTransposition(str1, str2);
227
228     Console.WriteLine($"\\n✓ Расстояние Левенштейна: {distance}");
229     Console.WriteLine($"✓ Расстояние Дameraу-Левенштейна (☐ перестановками): {distanceWithTransposition}");
230 }
231
232 static void WithMaxDistance()
233 {
234     Console.WriteLine("\n--- Проверка ☐ максимальным порогом ---");
235     Console.Write("Введите первую строку: ");
236     string? str1 = Console.ReadLine();

```

```

237     Console.Write("Введите вторую строку: ");
238     string? str2 = Console.ReadLine();
239     Console.Write("Введите максимальное допустимое расстояние: ");
240
241     if (str1 == null || str2 == null)
242     {
243         Console.WriteLine("Ошибка: некорректный ввод");
244         return;
245     }
246
247     if (int.TryParse(Console.ReadLine(), out int maxDistance))
248     {
249         int distance = LevenshteinCalculator.Calculate(str1, str2);
250         bool isValid = LevenshteinCalculator.IsWithinMaxDistance(str1, str2, maxDistance);
251
252         Console.WriteLine($"\\n✓ Расстояние Левенштейна: {distance}");
253         Console.WriteLine($"✓ Максимальное расстояние: {maxDistance}");
254
255         if (isValid)
256             Console.WriteLine($"✓ Строки совпадают (расстояние {distance} ≤ {maxDistance})");
257         else
258             Console.WriteLine($"X Строки ☐ совпадают (расстояние {distance} > {maxDistance})");
259     }
260     else
261     {
262         Console.WriteLine("Ошибка: введите корректное число");
263     }
264 }
265
266 static void ShowMatrix()
267 {
268     Console.WriteLine("\n--- Матрица расстояний ---");
269     Console.Write("Введите первую строку: ");
270     string? str1 = Console.ReadLine();

```

```

271 Console.WriteLine("Введите вторую строку: ");
272 string? str2 = Console.ReadLine();
273
274 if (str1 == null || str2 == null)
275 {
276     Console.WriteLine("Ошибка: некорректный ввод");
277     return;
278 }
279
280 Console.WriteLine();
281 Console.WriteLine(LevenshteinCalculator.MatrixToString(str1, str2));
282
283 int distance = LevenshteinCalculator.Calculate(str1, str2);
284 Console.WriteLine($"✓ Итоговое расстояние Левенштейна: {distance}");
285 }
286
287 static void BatchTest()
288 {
289     Console.WriteLine("\n--- Пакетное тестирование ---\n");
290
291     var testCases = new List<(string, string, int)>
292     {
293         ("kitten", "sitting", 3),
294         ("saturday", "sunday", 3),
295         ("book", "back", 2),
296         ("", "abc", 3),
297         ("abc", "", 3),
298         ("abc", "abc", 0),
299         ("fast", "cats", 3),
300         ("ant", "aunt", 1)
301     };
302
303     int passed = 0;
304     int failed = 0;

```

```

305
306     foreach (var (str1, str2, expected) in testCases)
307     {
308         int distance = LevenshteinCalculator.Calculate(str1, str2);
309         bool success = (distance == expected);
310
311         if (success)
312         {
313             Console.WriteLine($"✓ PASS: \"{str1}\" -> \"{str2}\" = {distance} (ожидалось {expected})");
314             passed++;
315         }
316         else
317         {
318             Console.WriteLine($"X FAIL: \"{str1}\" -> \"{str2}\" = {distance} (ожидалось {expected})");
319             failed++;
320         }
321     }
322
323     Console.WriteLine("\n--- Результаты ---");
324     Console.WriteLine($"Пройдено: {passed}/{testCases.Count}");
325     Console.WriteLine($"Провалено: {failed}/{testCases.Count}");
326 }
327 }
328
329

```

Результат работы программы

```
anya@Ultrabook12978:~/Term3/lab5$ dotnet run
=== Лабораторная работа №5: Расстояние Левенштейна ===

--- МЕНЮ ---
1. Простое вычисление расстояния
2. Проверка с максимальным порогом
3. Показать матрицу расстояний
4. Пакетное тестирование
5. Выход

Выберите действие: 1

--- Простое вычисление расстояния ---
Введите первую строку: cvbnp
Введите вторую строку: vcdfg

✓ Расстояние Левенштейна: 5
✓ Расстояние Дamerau-Левенштейна (с перестановками): 4

--- МЕНЮ ---
1. Простое вычисление расстояния
2. Проверка с максимальным порогом
3. Показать матрицу расстояний
4. Пакетное тестирование
5. Выход

Выберите действие: 2

--- Проверка с максимальным порогом ---
Введите первую строку: 12345
Введите вторую строку: 13333
Введите максимальное допустимое расстояние: 3

✓ Расстояние Левенштейна: 3
✓ Максимальное расстояние: 3
✓ Строки совпадают (расстояние  $3 \leq 3$ )

--- МЕНЮ ---
1. Простое вычисление расстояния
2. Проверка с максимальным порогом
```


3. Показать матрицу расстояний
4. Пакетное тестирование
5. Выход

Выберите действие: 3

--- Матрица расстояний ---

Введите первую строку: trtr

Введите вторую строку: trww

Матрица расстояний Левенштейна:

	t	r	w	w
0	1	2	3	4
t	1	0	1	2
r	2	1	0	1
t	3	2	1	1
r	4	3	2	2

✓ Итоговое расстояние Левенштейна: 2

--- МЕНЮ ---

1. Простое вычисление расстояния
2. Проверка с максимальным порогом
3. Показать матрицу расстояний
4. Пакетное тестирование
5. Выход

Выберите действие: 4

--- Пакетное тестирование ---

- ✓ PASS: "kitten" -> "sitting" = 3 (ожидалось 3)
- ✓ PASS: "saturday" -> "sunday" = 3 (ожидалось 3)
- ✓ PASS: "book" -> "back" = 2 (ожидалось 2)
- ✓ PASS: "" -> "abc" = 3 (ожидалось 3)
- ✓ PASS: "abc" -> "" = 3 (ожидалось 3)
- ✓ PASS: "abc" -> "abc" = 0 (ожидалось 0)
- ✓ PASS: "fast" -> "cats" = 3 (ожидалось 3)
- ✓ PASS: "ant" -> "aunt" = 1 (ожидалось 1)

```
✓ PASS: "kitten" -> "sitting" = 3 (ожидалось 3)
✓ PASS: "saturday" -> "sunday" = 3 (ожидалось 3)
✓ PASS: "book" -> "back" = 2 (ожидалось 2)
✓ PASS: "" -> "abc" = 3 (ожидалось 3)
✓ PASS: "abc" -> "" = 3 (ожидалось 3)
✓ PASS: "abc" -> "abc" = 0 (ожидалось 0)
✓ PASS: "fast" -> "cats" = 3 (ожидалось 3)
✓ PASS: "ant" -> "aunt" = 1 (ожидалось 1)
```

--- Результаты ---

Пройдено: 8/8

Провалено: 0/8

--- МЕНЮ ---

1. Простое вычисление расстояния
2. Проверка с максимальным порогом
3. Показать матрицу расстояний
4. Пакетное тестирование
5. Выход

Выберите действие: 5

Выход из программы...